

DISCOVERY PHASE – COMPLETE (Zero assumptions; requirements locked verbatim from your query + full Sovereign Machina architecture + changelog v3.2.7 + latest repo state)

Core functionality required:

- Mine the logic from Droid Computers (persistent, stateful agent runtime with filesystem, credentials, remote orchestration, always-on background execution, and session continuity).
- Determine if we can build a **similar function** on top of the existing Hermes supervisor.
- Choose **Rust or Go**, from scratch, while preserving the current tri/quad-node invariants (Node A = Mooncake KV buffer, Node B = Director, Node C = Oracle / Voice Core with Omi + 500 GB SSD, optional Hermes Pod).
- Focus on standalone personal intelligence use (voice-first, knowledge OS, decision support, life logging, self-improving agents).
- Maintain: zero-trust, physical sovereignty, declarative Nix identity, local-first design, no changes to Node A or Node B core logic.

All requirements are now exhaustive and production-locked.

RESEARCH PHASE – RIGOROUS CROSS-CHECK (Droid Computer logic + Sovereign Machina alignment)

From the referenced X post and FactoryAI ecosystem, Droid Computers provide:

- Persistent filesystem and credential store per agent instance.
- Stateful, resumable execution (tasks survive reboots, build on previous sessions).
- Remote orchestration (queue tasks from phone, monitor progress).
- Always-on background agents (triage alerts, run deep research).
- Session continuity (agents “get smarter the more they work”).

Your existing Hermes already does trajectory capture, intent enrichment, shard distillation, and self-healing. We can layer a native “Sovereign Droid Runtime” on top of Hermes without duplicating effort.

RECOMMENDATION

Yes — we can (and should) build a similar function on top of Hermes.

Language choice: Rust (preferred).

Approach: From scratch as a native Rust service layered on Hermes, not a fork or direct port.

Why Rust?

- Aligns with your existing Rust sidecars (artery daemon, Sovereign Observer).
- Memory safety and zero-cost abstractions are critical for a long-running, stateful agent runtime.
- Excellent async ecosystem (tokio) for persistent sessions and background tasks.
- Performance for filesystem, credential handling, and durable job queues.
- Go would be faster to prototype but less safe for long-running stateful workloads.

HIGH-LEVEL ARCHITECTURE – SOVEREIGN DROID RUNTIME (ON HERMES POD)

Core Components (Rust Crate: `sovereign-droid-runtime`)

Rust

```
// sovereign-droid-runtime/src/lib.rs
pub struct DroidRuntime {
    hermes: HermesHandle,           // Existing Hermes supervisor
    fs: PersistentFs,              // Isolated, versioned filesystem per
agent session
    vault: SovereignVault,         // Lightweight credential proxy (AES-
256 + steganography)
    queue: MinionQueue,            // Durable, resumable job queue
(SQLite-backed)
    state: SessionStore,           // In-memory + Mooncake spillover for
session continuity
}

impl DroidRuntime {
    pub async fn new(config: DroidConfig) -> Result<Self>;

    // Voice / intent from Node C
    pub async fn enqueue_voice_task(&self, note: VoiceNote) ->
Result<JobId>;

    // Background execution
    pub async fn run_minion(&self, task: MinionTask) -> Result<JobResult>;

    // Remote orchestration (from Machina Terminal via MCP)
    pub async fn get_session_status(&self, session_id: SessionId) ->
Result<SessionStatus>;
}
```

Key Features Built on Hermes

- **Persistent Filesystem:** Per-session isolated directories on the Hermes Pod's NVMe (or mounted volume). Uses git under the hood for versioned history (aligns with your RKG git model).
- **Credential Handling:** Leverage your existing steganography (ROOTS / GLOSSOPETRAE) + lightweight vault (extracted from Agent-Vault logic). Credentials never reach Hermes context — injected at execution time.
- **Durable Job Queue (Minions):** SQLite-backed queue with resumable subagents. Hermes supervisor owns the queue but delegates execution to the Droid runtime.

- **Session Continuity:** State stored in-memory with Mooncake spillover to Node A. Sessions survive restarts and build on each other (exactly like Droid Computers).
- **Remote Orchestration:** Machina Terminal (or phone via VPN) sends tasks via MCP; status and results stream back.
- **Self-Healing Tie-in:** Ouroboros audits every job and session; GEPA can evolve Droid runtime prompts and skills.

Integration Flow

1. Voice note arrives at Node C (Omi backend).
2. Hermes enriches + validates intent.
3. If background execution is needed, Hermes forwards to DroidRuntime on the Hermes Pod.
4. DroidRuntime queues the job, runs it in isolation, and reports back via MCP.
5. Results are ingested back into MemPalace / Akashik Artery.

Lightweight & Performant

- Runs as a single Rust binary on the Hermes Pod.
- Minimal memory footprint (in-memory sessions + spilling to disk).
- Leverages your existing Rust ecosystem — no new heavy dependencies.

ARCHITECT SIGN-OFF

Building a Sovereign Droid Runtime on top of Hermes in Rust is a natural and powerful extension. It gives you persistent, stateful agent execution, durable job queuing, session continuity, and remote orchestration while reusing your existing steganography, Parseltongue, Ouroboros, and GEPA layers. The runtime stays fully local-first, sovereign, and integrated into the Hermes Pod. This deepens the self-improving assistant without duplicating effort or compromising any invariants. Rust is the right choice for safety and performance in a long-running stateful runtime.

Your main sovereign machine remains completely untouched and rock-solid.

Clarifying questions (mandatory before next phase):

1. Do you want the starter Rust crate structure (`sovereign-droid-runtime`) with the key modules (PersistentFs, MinionQueue, SovereignVault)?
2. Shall I include the Nix module for running the Droid runtime on the Hermes Pod?

Reply with your priority and I will deliver the concrete starter code. The Droid-like runtime is ready to be prototyped on top of Hermes.